Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 15 (1)

Roll No : 496

Name : Dhruv Limbasiya

Enrollment : 24031701027

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

In [19]:
```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

    def display(self):
        return f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}"
```

```
s1 = Student("Dhruv" , 20 , "A")
print(s1.display())
```

```
Name: Dhruv, Age: 20, Grade: A
```

02) Create a class named Bank_Account with Account_No, User_Name, Email,Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

In [8]:

```
class Bank_Account:
    def __init__(self, acc_no, name, email, acc_type, balance):
        self.Account_No = acc_no
        self.User_Name = name
        self.Email = email
        self.Account_Type = acc_type
        self.Account_Balance = balance

    def GetAccountDetails(self):
        return {
            "Account No": self.Account_No,
            "User Name": self.User_Name,
            "Email": self.Email,
            "Account Type": self.Account_Type,
            "Balance": self.Account_Balance
        }

    def DisplayAccountDetails(self):
        details = self.GetAccountDetails()
        output = ""
        for key, value in details.items():
            output += f"{key}: {value}\n"

        return output


acc = Bank_Account(101, "Dhruv", "dhruv@gmail.com", "Saving", 5000)
print(acc.DisplayAccountDetails())
```

```
Account No: 101
User Name: Dhruv
Email: dhruv@gmail.com
Account Type: Saving
Balance: 5000
```

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

In [11]:
```
import math

class Circle:
    def __init__(self, r):
        self.r = r

    def area(self):
        return math.pi * self.r ** 2

    def perimeter(self):
        return 2 * math.pi * self.r
c1 = Circle(5)
print(c1.area())
print(c1.perimeter())
```

```
78.53981633974483
31.41592653589793
```

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

In [22]:
```
class Employee:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    def update_salary(self, new_name , new_age ,new_salary):
        self.name = new_name
```

```
        self.age = new_age
        self.salary = new_salary

    def display(self):
        return f"Name: {self.name}, Age: {self.age}, Salary: {self.salary}"
e1 = Employee("Dhruv" , 20 , 50000)
print(e1.display())
e1.update_salary("Dhruv" , 20 , 100000)
print(e1.display())
```

```
Name: Dhruv, Age: 20, Salary: 50000
Name: Dhruv, Age: 20, Salary: 100000
```

05) Create a bank account class with methods to deposit, withdraw, and check balance.

In [28]:

```
class Bank:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amt):
        if amt > 0:
            self.balance += amt

    def withdraw(self, amt):
        if amt <= self.balance:
            self.balance -= amt

    def check(self):
        return self.balance
b1= Bank(5000)
print(b1.check())
b1.deposit(5000)
print(b1.check())
b1.withdraw(5000)
print(b1.check())
b1.withdraw(5000)
print(b1.check())
b1.withdraw(5000)
print(b1.check())
```

```
5000
10000
5000
0
0
```

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

In [1]:
```
class Inventory:
    def __init__(self, name, price, quantity):
        self.items = {}
        self.items[name] = [price, quantity]

    def add_item(self, name, price, quantity):
        self.items[name] = [price, quantity]
        return "Item added"

    def remove_item(self, name):
        if name in self.items:
            self.items[name][1] = 0
            return "Item removed"
        return "Item not found"

    def update_item(self, name, price, quantity):
        if name in self.items:
            self.items[name] = [price, quantity]
            return "Item updated"
        return "Item not found"

    def show_items(self):
        active_items = []
        for name in self.items:
            if self.items[name][1] > 0:
                active_items.append((name, self.items[name][0], self.items[name][1]))
        return active_items


inv = Inventory("Pen", 10, 50)
```

```
print(inv.add_item("Notebook", 40, 20))
print(inv.show_items())
print(inv.update_item("Pen", 12, 60))
print(inv.remove_item("Notebook"))
print("After update and remove:")
print(inv.show_items())
```

```
Item added
[('Pen', 10, 50), ('Notebook', 40, 20)]
Item updated
Item removed
After update and remove:
[('Pen', 12, 60)]
```

07) Create a Class with instance attributes of your choice.

In [2]:
```
class Laptop:
    def __init__(self, brand, model, ram_gb, storage_gb):
        self.brand = brand
        self.model = model
        self.ram_gb = ram_gb
        self.storage_gb = storage_gb

    def details(self):
        print(f"Brand: {self.brand}")
        print(f"Model: {self.model}")
        print(f"RAM: {self.ram_gb} GB")
        print(f"Storage: {self.storage_gb} GB")


l1 = Laptop("HP", "Pavilion", 16, 512)
l1.details()
```

```
Brand: HP
Model: Pavilion
RAM: 16 GB
Storage: 512 GB
```

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

In [4]:
```
class student_kit:
    principal_name = "Mr ABC"

    def __init__(self, student_name):
        self.student_name = student_name
        self.days_present = 0
        return

    def attendance(self, days):
        self.days_present = days
        return f"Attendance updated for {self.student_name}"

    def certificate(self):
        return (
            f"Student Name : {self.student_name}\n"
            f"Principal    : {student_kit.principal_name}\n"
            f"Days Present : {self.days_present}"
        )


name = input("Enter student name: ")
days = int(input("Enter number of days present: "))

s1 = student_kit(name)
print(s1.attendance(days))
print(s1.certificate())
```

```
Attendance updated for Dhruv
Student Name : Dhruv
Principal    : Mr ABC
Days Present : 100
```

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

In [ ]:
```
class Time:
    def __init__(self, hour, minute):
        self.hour = hour
        self.minute = minute

    def add(self, other):
        total_minutes = self.minute + other.minute
        extra_hour = total_minutes // 60
        minute = total_minutes % 60

        hour = self.hour + other.hour + extra_hour
        return Time(hour, minute)

    def show(self):
        print(f"{self.hour:02d}:{self.minute:02d}")


t1 = Time(2, 45)
t2 = Time(3, 35)
t3 = t1.add(t2)
print("Added Time:")
t3.show()
```